

Comparing the predictability of pseudo- random number generators used in Python and Java by using a linear regression machine learning algorithm

Research Question: How does the average error margin in predictions, generated by a linear regression algorithm, vary between outcomes produced by the linear congruential generator, a PRNG utilized in Java's 'java.util.Random' class, and the Mersenne Twister, a PRNG employed in Python's 'random' module?

Word Count: 3982

Contents

1 Introduction.....	1
2 Background Information.....	3
2.1 Machine Learning.....	3
2.2 Linear Regression Machine Learning Algorithm.....	3
2.3 Cost function and Optimization Algorithms.....	4
2.4 Gradient Descent Optimization Algorithm.....	4
2.5 Learning rate and number of iterations in gradient descent.....	5
2.6 PRNG used in Java.....	6
2.7 PRNG used in Python.....	7
3 Methodology & Investigation.....	8
4 Data Presentation.....	9
5 Conclusion.....	16
5.1 Implication of findings.....	16
6 Limitations of Investigation & Improvements.....	17
7 Bibliography.....	19
8 Appendix.....	20
8.1 Java code used to generate random integers.....	20
8.2 Python code used to generate random integers.....	20
8.3 Linear Regression Algorithm used to train and test data values.....	21
8.4 List of software used.....	21

1. Introduction

Random number generators have been a very useful tool that have been used for various applications ranging from Cryptography to Gambling. Random numbers can be generated using physical methods such as rolling a die, and they can also be created by extracting randomness from physical phenomena that appear to be random such as atmospheric noise or a radioactive source. However, in most examples, different types of Pseudo-Random Number Generators (PRNGs), random number generators that rely on mathematical principles, are used to produce a sequence of random numbers, since random number generators that rely on physical phenomena are often memory intensive and slow compared to PRNGs.

These Pseudo-Random Number Generators do not demonstrate true randomness, instead, they produce a sequence of random numbers that pass the standard statistical test of randomness, which means that they exhibit properties such as uniform distribution and lack of correlation that one would expect from truly random sequences. However, because the numbers generated by a PRNG are not truly random and because these random numbers are created using an algorithm, a pattern that produces these random numbers must exist. If the pattern is found, then the values that are generated by a random number generator can become predictable.

Pseudo- Random number generators have a wide range of applications. They are used to determine random events in video games such as item drops, and they are often used in the initial stages of cryptographic algorithm development. Therefore, the predictability of these random number generators could pose significant risks and challenges. If a pseudo-random number generator (PRNG) is predictable, it can lead to vulnerabilities in many applications. For instance, in video games, if players are able to predict item drops or game outcomes, it

can lead to unfair advantages and disrupt the game's balance and integrity. This predictability can be exploited for cheating or gaining unauthorized benefits, ultimately harming the player experience and the game's reputation. Despite this, pseudo-random number generators are often used by game development companies because of their fast speed, and because it is very rare that people find out ways to exploit the game's predictability. Hence, the predictability of pseudo-random number generators is linked with real-world applications, making the topic of the investigation a worthy choice for the extended essay.

In recent times, machine learning has become a widely adopted method of detecting trends in data. Machine learning has been used in various applications in fields ranging from healthcare to autonomous vehicles. For this investigation, a machine learning algorithm will be used to determine the predictability of different pseudo-random number generators.

The predictability (the extent to which the sequence of numbers generated by the PRNG can be forecasted) of PRNGs typically used in two popular programming languages, Java and Python, will be compared to each other to find out which PRNG is the safer (least predictable) option when the implementation of a PRNG is inevitable. This topic of the essay will be discussed by exploring the research question "How does the average error margin in predictions, generated by a linear regression algorithm, vary between outcomes produced by the linear congruential generator, a PRNG utilized in Java's 'java.util.Random' class, and the Mersenne Twister, a PRNG employed in Python's 'random' module?" Error margins refer to the difference between the predicted value generated by the linear regression algorithm and the actual value generated by the PRNG. As expressed, the investigation aims to analyze which PRNG is more appropriate for generating a secure sequence of numbers that is difficult to predict.

2. Background Information

2.1 Machine learning

Machine Learning is the science of getting computers to act without being explicitly programmed (Stanford, n.d.). They are algorithms that make use of example datasets to extend their functionality. Machine Learning algorithms have become widespread in the use of self – driving cars, speech recognition, and chatbots in the past decade. In fact, it is so pervasive today that many people would have used it without even noticing it.

2.2 Linear Regression Machine Learning Algorithm

Linear regression provides a linear relationship between an independent variable and a dependent variable to predict the outcome of future events (Kanade, 2023). It is an example of a machine learning algorithm that is used in data science for predictive analysis (Kanade, 2023).

The regression model essentially predicts the value of the dependent variable from the independent variable by establishing a linear equation in the form of

$$Y = mX + c$$

Where:

Y = dependent variable

X = independent variable

m = the slope of the linear equation

c = the y-intercept of the line

The values of m and c are calculated by using something called a cost function, which will be discussed in the later parts of the essay. For understanding purposes, consider being tasked with estimating the price of different houses in a neighborhood without accessing any real estate listings. The most intuitive way to do this would be to consider features like the number of bedrooms, the number of bathrooms, the age of the house, and the proximity to the city center and estimate their market values based on these observations. Just as a person could estimate the market value (independent variable) of the house based on a number of characteristics (dependent variables), the Linear Regression algorithm learns to predict the independent variable by establishing relationships with dependent variables, resulting in a model that fits multiple variables to predict an outcome. A Multiple Linear Regression algorithm, an algorithm that considers two or more independent variables to calculate the dependent variable, will be used for this investigation.

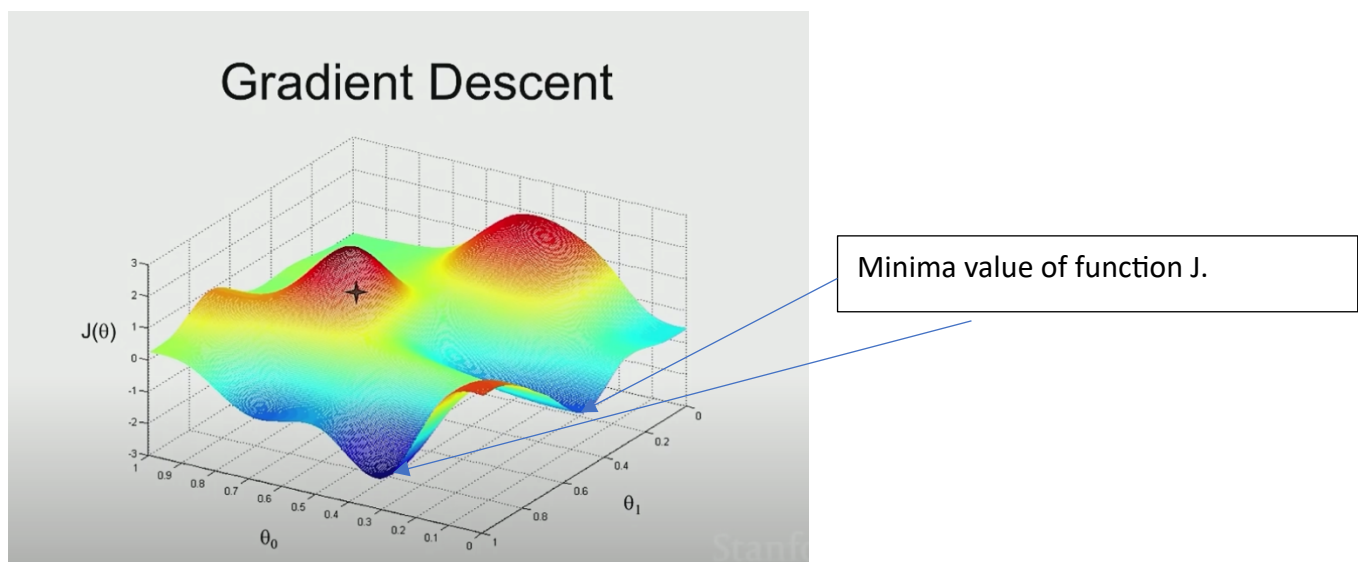
2.3 Cost Function and Optimization Algorithms

Linear regression machine learning algorithms rely on something called a cost function. A cost function is a measure of how well a machine learning model performs by quantifying the difference between the predicted and actual outputs (Shah, 2021). In other words, it tells us how well the model is doing by comparing the predicted values, generated by the machine learning algorithm, with the actual outputs in the data. Hence, minimizing the cost function would be synonymous with making the model perform better. There are many different optimization algorithms that are used to minimize the cost function. For this investigation, I will use the most widely used optimization algorithm, gradient descent optimization algorithm.

2.4 Gradient Descent Optimization Algorithm

To understand gradient descent, think of a cost function that has parameters from k_1 to k_n . An optimization algorithm aims to find the smallest value of the function. In other words, you want to pick values for k_1 to k_n that outputs the minimum number of J . To illustrate how a cost function works, at first, the cost function guesses the initial values of k_1 to k_n . The value that is outputted from the initial guess is then stored. Then, the parameters (k_1 to k_n) will be adjusted slightly each time, simultaneously, to attempt to minimize the function J . This adjustment of values will be repeated until the minimum of the cost function is found.

Figure 1. A visual representation of a gradient descent algorithm (Ng, n.d)



2.5 Learning rate and number of iterations in gradient descent

Previously in this essay, I mentioned how in gradient descent, the different parameters will be adjusted slightly each time simultaneously to attempt to minimize the cost function. The learning rate refers to how big this “slight adjustment” is. If the learning rate is too small, it will take a long time before the program finds the minimum of the cost function (function J), as it will take many iterations for the algorithm to find the true minimum. If the learning rate

is big enough, the gradient descent algorithm will be able to find the minimum of the cost function in a relatively short period of time, as it will require a fewer number of iterations. However, when the learning rate is too big, the “slight adjustments” tend to be too big, and the true minimum value of the cost function gets skipped by the algorithm and it may never reach the true minimum. This phenomenon is called “overshoot” in machine learning. For this investigation, the learning rate was set to 0.001 and the number of iterations was fixed to 2000.

2.6 PRNG used in Java

Java is a high-level, class-based, object-oriented programming language that is designed to have as few implementation dependencies as possible. Although there are many methods of generating random numbers in Java, in this essay, I will be using the “java.util.Random” class to generate a sequence of random numbers. The “java.util.Random” class uses a linear congruential generator, and in a linear congruential generator, random numbers are generated using the equation,

$$x_{n+1} = (ax_n + c) \bmod m$$

Where:

a = multiplying factor

c = increment factor

x_0 = starting value (seed)

m = modulo

$m > a, c, x_0$

Hence, linear congruential generators (LCGs), generate random numbers by using a starting value (initial seed), which is typically determined by the system time, and inputting this

through an algorithm to produce a random number (Bandyopadhyay, 2023). The first random number that is generated from the initial seed is put into the algorithm to generate the second random number and so on. For these reasons, usually, a random number generator is easily predictable when the starting value of the sequence of numbers is known. In this investigation, the linear regression machine learning algorithm will predict the random number generator, without being given the initial value.

2.7 PRNG used in Python

Python is an interpreted, object-oriented, high-level programming language with dynamic semantics (Python Software Foundation, 2023). Python is by far the most widely used and taught object-oriented programming language in the world. Python, like Java, utilizes a PRNG to generate a sequence of random values. Contrary to the PRNG used in Java, Python utilizes the Mersenne Twister generator to produce random numbers. The Mersenne Twister generator is implemented through the ‘random’ module which will be used for this investigation.

The Mersenne Twister uses a complex mathematical formula based on Mersenne prime numbers, ‘a number of the form $2^p - 1$ where p is a prime number’ (Clapham, 2005). The algorithm starts with a seed value, which acts as the initial input for generating the sequence of random numbers. Like the LCG, the initial seed is typically derived from the system time. After a series of mathematical operations, a sequence of random numbers is generated.

3. Methodology

In order to investigate the predictability of the pseudo-random numbers generated by the Java’s linear congruential generator and Python’s Mersenne Twister, I had to generate the random numbers from both programming languages and store it in a text file to make the numbers accessible by the linear regression machine learning algorithm. I coded the PRNGs

(see appendix) and stored the numbers as a text file. The numbers in the text file were used to train the algorithm and test the predictability of the linear regression model (see appendix). The linear regression algorithm also outputted the difference between the actual value and the predicted value along the predicted values. Visual Studio Code was chosen as the integrated development environment (IDE) for this investigation. Also, the linear regression machine learning algorithm was coded with Python and the NumPy library was used to create the gradient descent function. I chose to use Visual Studio Code as my IDE as it was one of the few IDEs that supported both Python and Java. The linear regression algorithm made use of the gradient descent algorithm.

3.1 Investigation

This analysis involves comparing predicted values of the 25th column in two text files that consists of 400 rows and 25 columns of numbers, each generated by PRNGs used in Java and Python. The process entails training a linear regression algorithm using the preceding numbers in each row, omitting the 25th column, predicting its value, and calculating the error margin as the difference between the predicted and actual values. This procedure is repeated for every 25th column leading up to each row. Subsequently, the average error margin is computed for every 50th row, providing insights into the predictive accuracy and randomness qualities of the datasets.

For clarification purposes, an example will be illustrated. Let's say that two rows are being used as training sets. In the first row, all the numbers in the text file leading up to the number in the 25th column of the row will be used to train the linear regression algorithm. Then, the 25th column of the row will be omitted in the training procedure and the linear regression algorithm will predict the number in the 25th column. The difference between the predicted

value and the actual value of the 25th column, the error margin, will be calculated for the row. This process is iterated for the second row, similarly, training the algorithm with preceding data and predicting the 25th column value. The average error margin is then determined by computing the average difference between the predicted and actual values of the two rows, providing a measure of predictive accuracy for this specific set of training data. For this investigation, the average error margin will be calculated for every 50th row. It is important to note that as the number of rows used to calculate the average error margin increases, it means that the overall number of rows used as training data also increases since all preceding rows are used as training data. Therefore, we expect the average error margin to decrease as the number of rows used to calculate the error margin increases, because the training data increases, making the linear regression model more accurate.

There are other possible methods that could be used to answer my research question. I could have used another optimization algorithm such as the normal equation optimization algorithm. However, I decided to use a gradient descent algorithm for my investigation because of its simplicity and ease of implementation along with the fact that I was more familiar with the gradient descent algorithm.

3.2 Procedure in steps

1. Obtain two text files, each containing 400 rows with 25 columns of random numbers generated using Java's "java.util.Random" class and Python's 'random' module.
2. For each row:
 - Select all numbers leading up to the 25th column as training data.
 - Train a linear regression algorithm using the selected data, omitting the 25th column.
 - Predict the value of the 25th column using the trained model.

- Calculate the error margin as the difference between the predicted and actual values.
- 3. Repeat the above steps for every 25th column leading up to each row in the dataset.
- 4. Once error margins are calculated for all rows, compute the average error margin for every 50th row to provide a measure of predictive accuracy for the respective datasets.

4. Data Presentation

The average error margin for the corresponding number of rows used as training sets were recorded on a table. The tables below show averages, a form of processed data.

Table 1: Table showing the number of rows used to work out the error margins and their corresponding average error margin for the Mersenne Twister used in Python's "random" module

Number of Rows used to work out the error margins	Average Error margin (4 decimal places)
50	2.7940
100	2.3978
150	2.3807
200	2.3303
250	2.3076
300	2.2790
350	2.2515
400	2.2459

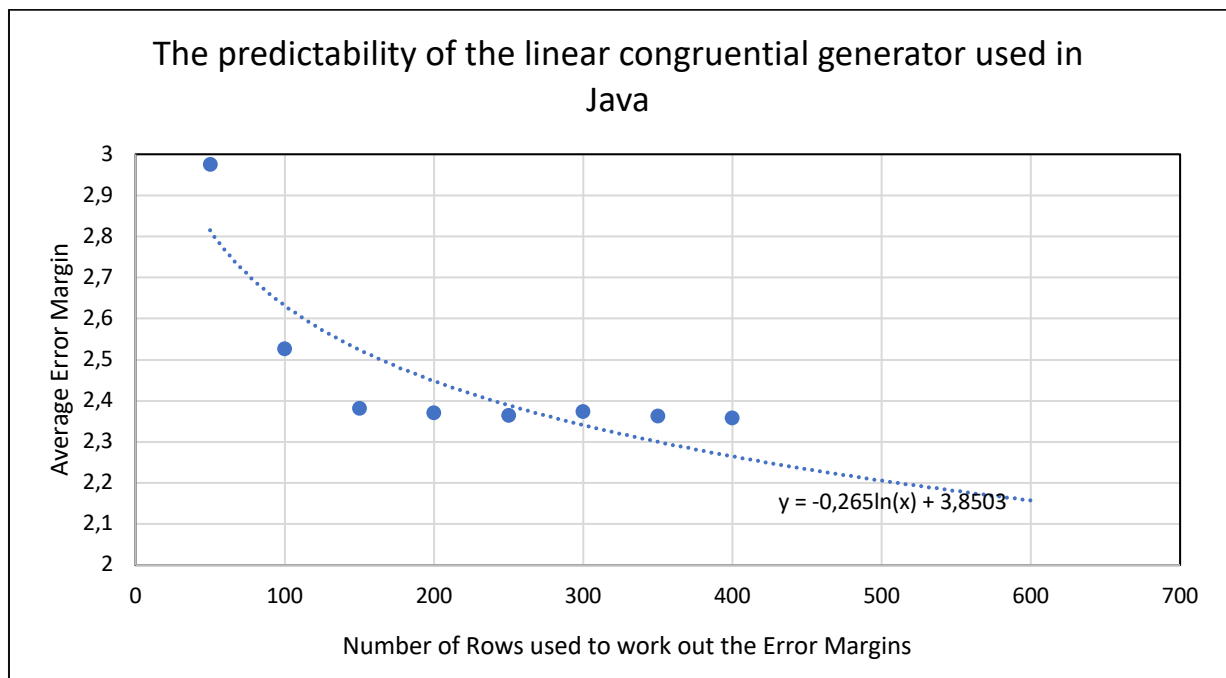
Table 2: Table showing the number of rows used to work out the error margins and their corresponding average error margin for the linear congruential generator used in Java's "java.util.Random" class

Number of Rows used to work out the error margins	Average Error margin (4 decimal places)
50	2.9756
100	2.5261
150	2.3819
200	2.3703
250	2.3639
300	2.3741
350	2.3637
400	2.3586

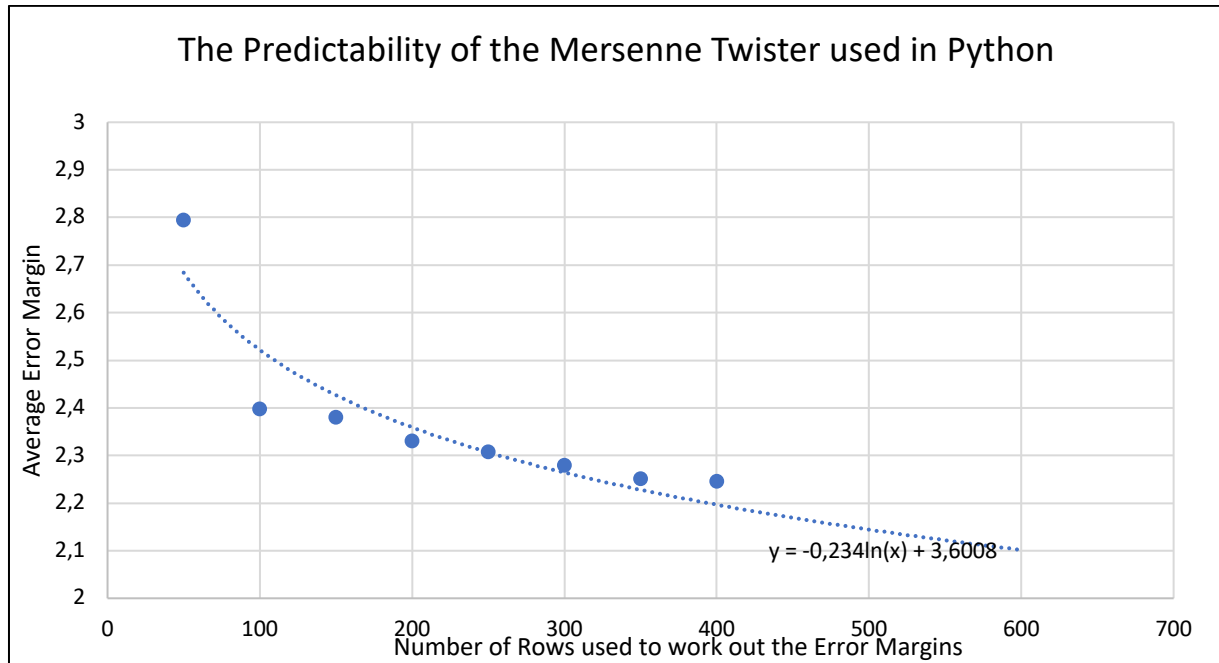
The tables 1 and 2 show the change in the average error margin as the number of rows used to work out the error margins increase. The average error margin was recorded up to four decimal places to achieve a high degree of precision in the investigation. For both tables, it showed that as the number of rows used to work out the error margins increased, the average error margin decreased, representing an inversely proportional relationship. As mentioned previously, this is because as the number of rows used to work out the average error margins

increase, the overall number of rows used for training data increases. To visualize this trend, the data recorded in the two tables were plotted into the following graphs .

Graph 1: Graph showing the number of rows used to work out the error margins and their corresponding average error margin for the linear congruential generator used in Java's "java.util.Random" class

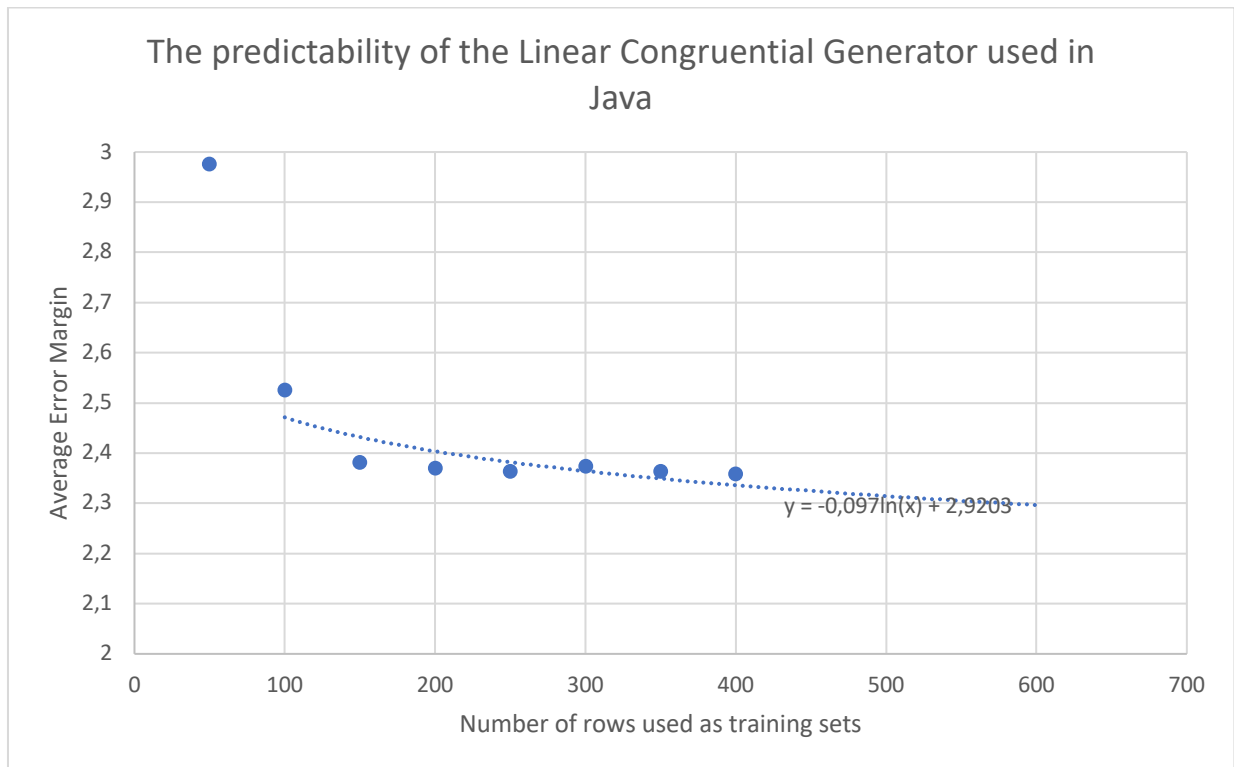


Graph 2: A graph showing the number of rows used to work out the error margins and their corresponding average error margin for the Mersenne Twister used in Python's "random" module

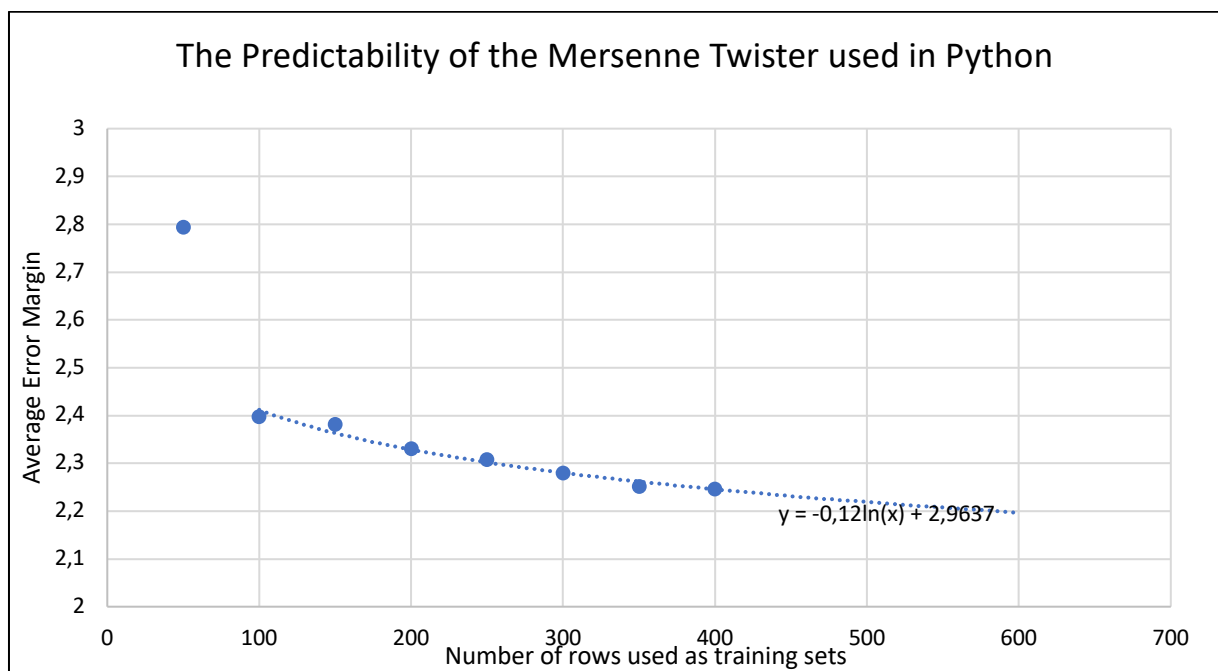


As you can see, when data is plotted from using 50 rows as the number of rows used to calculate the average margins, the overall trend of the curve is not representative of the relationship between the two variables. Hence, I had to take out the outlier, the average error margin when 50 rows were used as the number of rows used to work out the error margin, from the analysis to get two graphs as shown below.

Graph 3: Graph showing the number of rows used to work out the error margins and their corresponding average error margin for the linear congruential generator used in Java's "java.util.Random" class excluding the outlier



Graph 4: graph of average error margin plotted against the number of rows used as training sets for the linear regression algorithm for the set of random numbers generated by PRNG used in Python excluding the outlier



The two graphs plotted for both Java and Python's pseudo random number generator showed a logarithmic relationship. To find which pseudo-random number generator is more easily predicted by the linear regression machine learning algorithm, it would be ideal to calculate the minimum of the two curves.

The equation for the first graph (Graph 3) is:

$$y = -0.097 \ln(x) + 2.9203$$

And the equation for the second graph (Graph 4) is:

$$y = -0.12 \ln(x) + 2.9637$$

As you can see, the two graphs are logarithmic relationships that represent a function where y (the average error margin) continuously decreases as x (the number of rows used as to calculate the error margin) increases. Thus, a minimum value cannot be deduced from the two logarithmic graphs.

Therefore, an alternative method for determining which pseudo-random number generator is more easily predicted by the linear regression machine learning algorithm was developed. I created a table showcasing hypothetical average error margins calculated for a greater number of rows, utilizing the equations derived from the trendline analysis.

Table 3: Table of number of rows of data used to work out the error margins and their corresponding hypothetical average error margin for Java's "java.util.Random" class

Number of Rows used to work out the error margins	Hypothetical average error margin
2500	2.1614
5000	2.09413
7500	2.0548
10000	2.0269

Table 4: Table of number of rows of data used to work out the error margins and their corresponding hypothetical average error margin for Python's "random" module

Number of Rows used to work out the error margins	Hypothetical average error margin
2500	2.0248
5000	1.9416
7500	1.8930
10000	1.8585

It can be noticed that the hypothetical average error margins in table 4 are smaller than the smallest average error margin in table 3. Thus, using hypothetical average error margins from tables 3 and 4, it can be concluded that the set of pseudo random integers generated by Python's Mersenne Twister is more predictable than the set of pseudo-random integers generated by Java's LCG. However, the difference in the error margin is very small.

5. Conclusion

For both PRNGs used in the two programming languages, they showed a logarithmic relationship where the average error margin decreased as the number of rows used as training sets increased. The findings discovered in this investigation show that although a smaller average error margin could be achieved by training the linear regression algorithm with more training sets, the decrease in the average error margin significantly plateaus around 2 for both PRNGs used in both programming languages. However, it was shown that the Python's Mersenne Twister, which is the PRNG used in python's random module, was easier to predict. To answer the research question, the average error margin when predicting the outcome of the PRNG, was smaller for the PRNG employed in Python's 'random' module

than the PRNG used in Java's "java.util.Random" class. Hence, it can be deduced that when choosing a more secure pseudo random integer generator, it would be wise to choose the linear congruential generator, which is the PRNG used in Java's 'java.util.Random' module.

5.1 Implication of findings

The implications of the findings in this investigation show that using the linear congruential generator, which is used in Java's 'java.util.Random' module, is a more secure option when generating a set of pseudo-random numbers. However, while linear congruential generators are more secure, the Mersenne twister generator tends to be at least two times faster than the linear congruential number generators (Jindal, 2021). Nevertheless, it can be argued that as processing techniques become more efficient and operate faster, this factor becomes meaningless in choosing which type of PRNG to use. Hence, there are different benefits and drawbacks of using different methods of generating pseudo-random integers.

The implications of the findings in this investigation could be used in real-life applications. One such application is in the development of an application that makes use of randomness, such as online UNO. True random generators are complicated to compute and often very slow as it doesn't rely on any computational algorithms. Hence, the use of a PRNG is often inevitable when coding an online application such as online UNO. A user could use a machine learning algorithm such as a linear regression algorithm to predict the next card to be drawn in the game, based on the sequence of previous cards. This would lead to the user gaining an unfair edge against other players in the game. Using the findings discussed in the investigation, when the game developer has the choice of using a PRNG through Java's "java.util.Random" class or Python's "random" module, the game developer could choose to

use a pseudo-random integer generator that is less predictable, in this case the linear congruential generator, to minimize the unfair edge gained by the user.

However, if an application is targeted at people using devices that are often slow and tends to perform poorly when dealing with memory-intensive operations, it could be a better decision to use a PRNG that is faster, making it more optimal to use the Mersenne twister that is used in Python's 'random' module, since the difference in predictability is small.

6. Limitations of Investigation & Improvements

Although the investigation aimed to answer the research question as accurately as possible, there were some limitations to this investigation. The limitations of the investigation include the following:

1. Only 400 rows and 25 columns of pseudo random number were used to draw a trendline in this investigation. A larger dataset would better simulate real-life applications when predicting random numbers as hackers and people that attempt to exploit the predictability of a PRNG will likely use a much larger amount of training sets when attempting to beat casino games or disrupt the game's balance. This could have been improved in our investigation by increasing the number of times our dependent variable, the average error margin, was measured, and increasing our independent variable, the number of rows used as training sets and its interval.
2. The PRNGs used in both programming languages were asked to generate integers from 0 to 9. This makes the investigation partly inapplicable to real-world scenarios as it is often the case that Pseudo-random number generators are asked to generate numbers that have more digits. The investigation was set up this way because a higher range of values would increase the error margin and make the linear regression

algorithm harder to predict the “random” number, resulting in the need of a bigger amount of training data which takes up a lot of time to generate. This limitation could have been improved by increasing the range of values, perhaps 0 to 99, and increasing the number of rows used as training sets.

3. Only random integers were programmed to be outputted by the PRNGs. This again makes the investigation partly inapplicable to real-world scenarios as it is often the case that Pseudo-random number generators are asked to generate numbers that include decimals. This investigation was designed in a way that the training data did not include numbers that had decimals as I thought this could make the numbers more difficult to predict, which necessitates the acquisition of a larger volume of training data, leading to significant time consumption during the generation process. This limitation could have been improved by including decimals in the training set.

7. Bibliography

- Arbeláez, O.D. (2016). *How competitive are C++ standard random number generators*. [online] Medium. Available at: <https://medium.com/@odarbelaeze/how-competitive-are-c-standard-random-number-generators-f3de98d973f0#:~:text=The%20author%20of%20the%20question> [Accessed 18 Dec. 2023].
- Jindal, I. (2021). *How to generate random numbers?(Linear Congruential Generator)*. [online] Medium. Available at: <https://ij007.medium.com/how-to-generate-random-numbers-linear-congruential-generator-c46ac543fe8a> [Accessed 18 Dec. 2023].
- Kanade, V. (2023). *What Is Linear Regression? Types, Equation, Examples, and Best Practices for 2022* |. [online] Spiceworks. Available at: <https://www.spiceworks.com/tech/artificial-intelligence/articles/what-is-linear-regression/#:~:text=Linear%20regression%20is%20an%20algorithm>.
- Marji, B. (n.d.). *What is Mersenne Twister?* [online] Educative: Interactive Courses for Software Developers. Available at: <https://www.educative.io/answers/what-is-mersenne-twister>.
- mlclass.stanford.edu. (n.d.). *Machine Learning - Stanford University*. [online] Available at: <http://mlclass.stanford.edu/#:~:text=Machine%20learning%20is%20the%20science> [Accessed 18 Dec. 2023].
- Python Software Foundation (2023). *What is Python? Executive Summary*. [online] Python. Available at: <https://www.python.org/doc/essays/blurb/>.
- Shah, S. (2021). *Cost Function | Types of Cost Function Machine Learning*. [online] Analytics Vidhya. Available at: <https://www.analyticsvidhya.com/blog/2021/02/cost-function-is-no-rocket-science/>.
- Bandyopadhyay, A. (2023). *Is it safe to use java.util.Random to generate random key ?* [online] DEV Community. Available at: <https://dev.to/arpanforgeek/is-it-safe-to-use-javautilrandom-to-generate-random-key--5cmj#:~:text=in%20sensitive%20applications%3A-> [Accessed 12 Dec. 2023].

Clapham, C. and Nicholson, J.R. (2005). *The concise Oxford dictionary of mathematics*. Oxford: Oxford University Press.

Ng, A. (n.d.). *1.2.5 Gradient Descent by Andrew Ng*. [online] [www.youtube.com](https://www.youtube.com/watch?v=yFPLyDwVifc&ab_channel=ComputerScienceEngineering). Available at:
https://www.youtube.com/watch?v=yFPLyDwVifc&ab_channel=ComputerScienceEngineering [Accessed 12 Jan. 2024].

8. Appendix

8.1 Java code used to generate random integers

```

1  import java.io.BufferedWriter;
2  import java.io.FileWriter;
3  import java.io.IOException;
4  import java.util.Random;
5
6  public class RandomIntMatrixGenerator {
7
8      Run | Debug
9      public static void main(String[] args) {
10         int rows = 400;
11         int columns = 25;
12         String fileName = "random_int_matrix_400x25.txt";
13
14         // Generate and write the random matrix to a file
15         writeRandomIntMatrixToFile(rows, columns, fileName);
16         System.out.println("Matrix generated and written to " + fileName);
17     }
18
19     private static void writeRandomIntMatrixToFile(int rows, int columns, String fileName) {
20         Random random = new Random();
21
22         try (BufferedWriter writer = new BufferedWriter(new FileWriter(fileName))) {
23             for (int i = 0; i < rows; i++) {
24                 for (int j = 0; j < columns; j++) {
25                     int randomInt = random.nextInt(bound:10);
26                     writer.write(Integer.toString(randomInt));
27                     if (j < columns - 1) {
28                         writer.write(str:", ");
29                     }
30                 }
31                 writer.newLine();
32             }
33         } catch (IOException e) {
34             e.printStackTrace();
35         }
36     }
37 }

```

Source: Author

8.2 Python code used to generate random integers

```

1  import random
2  random_matrix = [[random.randint(0, 9) for _ in range(25)] for _ in range(400)]
3  print(random_matrix)
4

```

Source: Author

8.3 Linear Regression Algorithm used to train and test data values.

```

1  import numpy as np
2
3
4  data = np.genfromtxt('/Users/yeoyejun/Downloads/EEjavanum.txt', delimiter=', ')
5
6  X = data[:, :-1]
7  y = data[:, -1]
8
9
10 X = np.concatenate([np.ones((X.shape[0], 1)), X], axis=1)
11
12
13 theta = np.zeros(X.shape[1])
14
15
16 alpha = 0.001
17 num_iters = 20000
18
19
20 def gradient_descent(X, y, theta, alpha, num_iters):
21     m = len(y)
22     J_history = np.zeros(num_iters)
23
24     for iter in range(num_iters):
25         errors = np.dot(X, theta) - y
26         theta = theta - (alpha / m) * np.dot(X.T, errors)
27         J_history[iter] = (1 / (2 * m)) * np.sum(errors ** 2)
28
29     return theta, J_history
30
31
32 X_train = X[:400, :]
33 y_train = y[:400]
34
35
36 theta, J_history = gradient_descent(X_train, y_train, theta, alpha, num_iters)
37
38 predicted = np.dot(X, theta)
39
40
41 result_table = np.column_stack((np.arange(1, X.shape[0]+1), y, predicted, predicted - y))
42
43
44 print('Row Number | Expected 25th Number | Gradient Descent Predicted 25th Number | Error margin')
45 print(result_table[0:0:])
46
47
48 average_error_margin = np.mean(np.abs(result_table[:, 3]))
49
50 print('Average Error Margin:')
51 print(average_error_margin)

```

Source: Author

8.4 List of software used

Software used to generate random numbers and train the linear regression model :

- Visual Studio Code